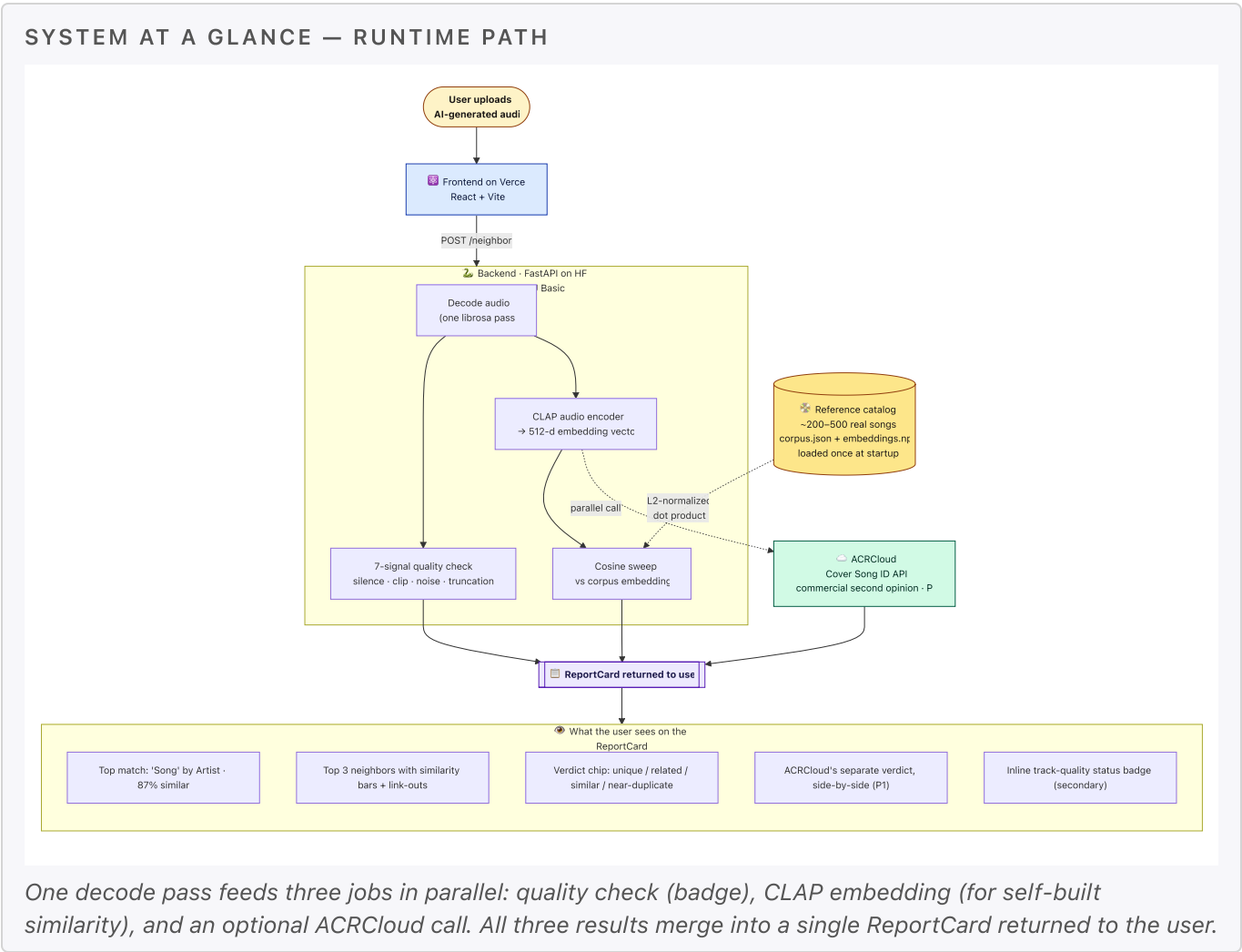


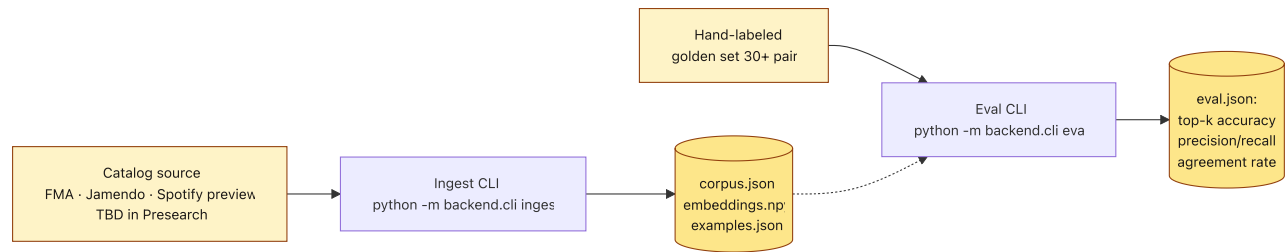
PRD — PiedPiper

Pre-publish acoustic-similarity scanner for AI-generated music.
Upload a track → closest real-song match + similarity % + top-3 neighbors + risk verdict.

status: Complete · updated: 2026-06-04 · audience: warm-intro read by Suno's Head of Engineering



SYSTEM AT A GLANCE — OFFLINE PIPELINES



These run on your laptop (or CI), not at request time. The artifacts they produce — the corpus and the eval — are committed to the repo and read by the live app at startup or via the eval page.

PRD — PiedPiper

Pre-publish acoustic-similarity scanner for AI-generated music. Upload a track → closest real-song match + similarity % + top-3 neighbors + risk verdict.

Overview

PiedPiper is a deployed web app. A user drops an audio file in; the app returns a one-screen report dominated by acoustic-similarity output (closest real song + score + top-3 neighbors + risk verdict) with a secondary track-quality status badge. A separate evaluation page reports measured detector quality against a hand-labeled golden set.

The codebase inherits ~90% of its audio-ML engineering from the prior Soundcheck work in `backend/backend/`. The build focuses on the new pieces: reference catalog, similarity-first UI, hybrid second opinion (ACRCloud), similarity eval, and PiedPiper rebranding.

Goals

1. **Working similarity check.** An uploaded AI track returns a meaningful closest real-song match with a similarity percentage in ≤ 5 seconds (warm path).
2. **Substance for a warm-intro read.** A reviewer who spends 5–20 minutes on the project (the actual reading mode for this audience) can trace the engineering judgment all the way through: catalog choice, embedding choice, threshold calibration, eval results, what's deliberately out of scope.
3. **Honest evaluation.** The eval page presents top-k retrieval accuracy, per-verdict precision/recall, and labeled false-positive/false-negative examples against a hand-curated golden set.
4. **Two-system comparison.** ACRCloud Cover Song Identification runs alongside the self-built CLAP pipeline; both verdicts shown side-by-side with an agreement metric on the eval page. *(Was a stretch goal before; promoted because the side-by-side comparison is the single most-substantive engineering signal in the project.)*
5. **README / about page that earns the read.** Written for the actual audience: walks the reader through architecture, the catalog-source decision, threshold calibration methodology, eval results, and an explicit "what I deliberately left out and why" section.
6. **Cost discipline.** Hosting stays ~\$0 (Vercel + free HF Space CPU Basic). Commercial API usage stays within a stated monthly budget.

Users

- **Real audience: Suno's Head of Engineering.** Warm-intro reader; 5–20 minute substantive review. Engages with depth and conviction.
- **Framing-device audience: platform-side operators** (ML Eval, T&S, Platform Engineering). Not the actual reader; the mental model the README invites.
- **Stand-in: any visitor** who opens the deployed URL.

Core flows

Flow 1 — Upload and analyze (the headline)

1. User opens the deployed URL.
2. User drags an audio file onto the drop zone, or clicks to file-pick.
3. App shows an "analyzing..." state; copy swaps to a "warming up" message if the response is slow (HF Space cold-wake ~30 s).
4. App returns a ReportCard:
 - **Top match** — name + artist + similarity % + link out to the source platform.
 - **Top 3 neighbors** — similarity bars + link-outs.
 - **Verdict chip** — unique / related / similar / near-duplicate.
 - **Inline track-quality status badge** — "Track quality: ok" or "issues detected — click to expand".
 - **Expandable quality details** — the legacy 7-signal breakdown, demoted to a collapsed section.

Flow 2 — Try an example

1. User clicks an example chip on the landing page.
2. App displays the same ReportCard shape, populated with the precomputed result for that staged track (no live backend call).

Flow 3 — View evaluation page

1. User navigates to `/evaluation`.
2. Page shows: top-k retrieval accuracy, per-verdict precision/recall, threshold calibration, and a small set of labeled false-positive and false-negative examples.
3. Where the hybrid is enabled: agreement rate between the self-built CLAP pipeline and ACRCLOUD on the golden set.

Functional requirements

Must-have (P0 — required for a credible v1)

1. **Frontend similarity flow.** Upload flow calls `POST /neighbors` and renders top match + top-3 + verdict from the response.
2. **Reference catalog.** `quality-scorer/public/corpus/corpus.json` + `embeddings.npy` exist with ≥ 200 tracks of real popular music, embedded with the same CLAP checkpoint used at query time.
3. **Backend `/neighbors` reuse.** No backend rewrite required — endpoint at `backend/backend/api.py:205-245` is kept as-is; thresholds at `:49-51` may be tuned post-calibration.
4. **PiedPiper renaming.** Page title, `quality-scorer/README.md`, `backend/backend/api.py` FastAPI title, nav header, and the directory name (if practical) all read PiedPiper.
5. **Quality status badge.** Inherited broken-output detector runs in the same decode pass and surfaces as a one-line inline badge on the ReportCard.
6. **Example chips.** Landing page resolves examples from a precomputed `examples.json` in the corpus directory.

Should-have (P1 — substantive engineering signal; warm-intro reader expects this)

7. **Similarity eval surface.** Evaluation page measures the similarity detector against a hand-labeled golden set (≥ 30 labeled query $\rightarrow$ nearest pairs), reporting top-k retrieval accuracy and per-verdict precision/recall, with named false-positive and false-negative examples.
8. **Reproducible catalog ingest.** Catalog can be rebuilt from a CLI command (`backend/backend/cli.py`), with the source pipeline documented in the README.
9. **ACRCloud second opinion.** Cover Song Identification API runs as a parallel column on the same upload; both verdicts shown side-by-side on the ReportCard.
10. **Agreement metric.** Eval page reports agreement rate between the self-built CLAP pipeline and ACRCloud across the golden set.
11. **README / about page.** Walks the reader through: architecture, why each technical choice was made, threshold calibration methodology, eval results summary, and a "what I deliberately left out" section. Written to be read by Suno's Head of Eng, not to be SEO copy.

Nice-to-have (P2 — polish, not blockers)

12. **Eval gating.** URL query param to gate the eval page behind `?dev` .
13. **Rights documentation.** README documents catalog rights and the "this is a sampled demo set" caveat plainly.
14. **Data-driven thresholds.** The 0.95/0.85/0.70 cutoffs are recalibrated from observed AI-vs-real distribution rather than carried over by assumption.

Non-functional considerations

- **Latency.** Warm response ≤ 5 s per upload (one decode pass shared across librosa quality, CLAP embed, cosine sweep). Cold HF Space wake ≤ 45 s; UI mitigates with copy swap at 6 s elapsed.
- **Cost.** \$0 hosting steady-state. ACRCloud cost capped at a stated monthly budget (locked in Decide).
- **Privacy.** Stateless — no persistence per visit. Uploaded audio not stored after the request returns.
- **Audio rights.** Catalog stores embeddings + metadata only; no source audio bytes shipped in the deployed app. Link-outs to source platform.
- **Compatibility.** mp3 primary; wav/flac/ogg/m4a accepted. 50 MB upload cap (already enforced).
- **Failure modes.** `unsupported_media` / `empty_file` / `file_too_large` / `decode_failed` / `empty_audio` handled cleanly (already wired). `no_corpus` state surfaces gracefully if catalog is unavailable.

Non-goals

Inherited from PROBLEM_SUMMARY:

- No music generation.
- No discovery / recommendation surfaces.
- No user accounts or persistence.
- No automation against Suno's web service.
- No claim of full-catalog coverage.

- No musical / aesthetic judgment.
- No exact-recording fingerprinting (Shazam-style).

Added at PRD level:

- **No "copyright detector" framing.** The product is positioned as a risk scanner; the verdict language is acoustic-similarity language (unique / related / similar / near-duplicate), not legal language (infringing / non-infringing).
- **No multi-track batch upload** in the demo.

Open questions (deferred to Presearch / Decide)

- **Catalog source.** Spotify previews (mid-deprecation), FMA, MTG-Jamendo, ACRCLOUD's own catalog, or a mix — needs Presearch.
 - **Catalog target size.** Provisional 200–500 tracks; revisit based on chosen source's yield.
 - **Similarity thresholds.** Current carry-over is 0.95/0.85/0.70; recalibrate after observing real AI-vs-real distribution on the golden set.
 - **Eval label rubric.** Binary "unacceptable copy yes/no" vs. 4-class verdict label.
 - **ACRCLOUD inclusion as P1.** Should be locked in Decide based on pricing inspection and observed golden-set agreement.
 - **Eval page visibility.** Public or gated behind `?dev`.
-

Appendix A — Presearch outline

Presearch resolves the unknowns this PRD intentionally deferred. To keep that phase clean, it is structured in **two passes**: logic first, tech stack second. The logic pass defines what the system has to *do*; the tech pass picks the specific tools that enable it. Doing them in this order prevents the tech discussion from drifting into "what's cool" instead of "what we need."

Pass 1 — Logic & app-flow details

System-internal questions. Independent of which library or service implements them.

- **Request-path internals.** For a `POST /neighbors` upload, what is the exact sequence — validation, decode, librosa quality, CLAP embed, cosine sweep, optional ACRCLOUD call, response assembly? What data shapes pass between each step?
- **Verdict decision tree.** Top similarity → verdict mapping (`unique` / `related` / `similar` / `near-duplicate`). Exact thresholds, edge cases at the boundaries, and what happens when no catalog match exceeds the lowest threshold.
- **Hybrid disagreement handling.** When CLAP says "near-duplicate" but ACRCLOUD says "no match" (or vice versa), what does the user see? Both verdicts side-by-side, the more conservative one, an "investigate" affordance?
- **Catalog freshness.** Is the catalog a static commit-time artifact, or does it update? How do we add a track? Do we re-embed everything if we swap CLAP checkpoints?
- **Quality-badge thresholds.** Which of the 7 quality signals surface inline on the badge, which only on expand? What counts as "ok" vs "issues detected"?

- **Eval methodology.** What counts as a "hit" in top-k retrieval — exact track, same artist, same song-family? How is the golden set constructed? What's the labeling rubric (binary vs 4-class)?
- **Failure surfaces.** For each error code (`unsupported_media` , `empty_file` , `file_too_large` , `decode_failed` , `empty_audio` , `no_corpus` , ACRCLOUD timeout/quota exceeded), what does the user actually see in the UI?
- **Example-chip behavior.** Are examples precomputed verdicts displayed instantly (no backend), or do they re-run the live pipeline each click? Trade-off: speed vs honesty.

Pass 2 — Tech stack to enable Pass 1

Once the logic is locked, these questions pick the specific tools. Each tech choice should trace back to a logic requirement from Pass 1.

- **Reference catalog source.** FMA, MTG-Jamendo, Spotify previews (mid-deprecation), ACRCLOUD's own catalog, or a hybrid mix. Drives recognizability of matches ("oh, Blinding Lights" lands; "track #4711 from MTG-Jamendo" doesn't), rights story, and the size we can realistically reach.
- **Audio embedding model.** Stay with the inherited music-tuned CLAP 512-d checkpoint, or evaluate alternatives (LAION-CLAP general, MERT, MuLan)? Inherited choice is the default unless something better surfaces.
- **Vector search method.** In-memory cosine sweep is adequate at ~500 tracks. If we ever scale to thousands, do we plan for FAISS / HNSW now or defer?
- **Commercial second opinion.** ACRCLOUD Cover Song ID — pricing tier, monthly budget cap, error-mode SLA, and what counts as a "match" in their response payload (their confidence score semantics).
- **Frontend stack.** Keep React + Vite, or revisit (Next.js, Astro)? Default: keep.
- **Backend hosting.** Keep Hugging Face Space CPU Basic, or move (Fly.io, Modal, Render)? Cold-start tolerance is the main trade-off.
- **Frontend hosting.** Keep Vercel, or move? CDN considerations if the corpus JSON grows.
- **CI / build.** GitHub Actions for ingest + eval CLI runs? Where does `eval.json` live (committed vs generated on deploy)?

Output of Presearch

A `PRESEARCH.md` artifact that, for each question above, captures:

- The 2–3 candidate answers considered.
- The trade-offs.
- The recommendation carried into the Decide phase.

This appendix is the **scaffold**; Presearch is where the actual research and recommendations get written.